Assignment 4 — Vertex Coloring and PageRank

1. Overview

This submission implements **only the two graph algorithms required for this Assignment 4 version**:

1. **Greedy Vertex Coloring using the Welsh–Powell ordering**
2. **PageRank**

Both algorithms operate on a **CSR (Compressed Sparse Row)** graph representation.

This final version does **not** implement:

- K-Means
- FastMap
- Assignment 3 MST algorithms (Kruskal / Prim)

The runtime driver therefore contains only Vertex Coloring and PageRank.

---

2. Project Structure

```
assignment_04/
│
├── driver/
│   └── main.cpp
│
├── src/
│   ├── graph.h
│   ├── graph_io.cpp
│   ├── color.h
│   ├── color.cpp
│   ├── pagerank.h
│   └── pagerank.cpp
│
├── tests/
│   ├── color/
```

```
|   |   ├── color_10.txt
|   |   ├── color_100.txt
|   |   ├── color_10000.txt
|   |   ├── color_50000.txt
|   |   └── color_100000.txt
|   |
|   └── pagerank/
|       ├── pagerank_10.txt
|       ├── pagerank_100.txt
|       ├── pagerank_1000.txt
|       ├── pagerank_10000.txt
|       └── pagerank_50000.txt
|
├── outputs/
|   ├── color/
|   └── pagerank/
|
├── Makefile
├── README.md
├── RESULTS.md
└── generate_tests.py
```

---

3. Source Files

3.1 `driver/main.cpp`

The driver is the runtime interface.

The user does **not** enter a test-file name manually.

The first menu is:

```
===== Assignment 4 =====
1. Vertex Coloring
2. PageRank
0. Exit
Enter choice:
```

After selecting an algorithm, the user enters only a number from `1` to `5`.

The driver automatically maps that number to the appropriate test file.

Vertex Coloring

Choice	File	V
1	`tests/color/color_10.txt`	10
2	`tests/color/color_100.txt`	100
3	`tests/color/color_10000.txt`	10,000
4	`tests/color/color_50000.txt`	50,000
5	`tests/color/color_100000.txt`	100,000

PageRank

Choice	File	V
1	`tests/pagerank/pagerank_10.txt`	10
2	`tests/pagerank/pagerank_100.txt`	100
3	`tests/pagerank/pagerank_1000.txt`	1,000
4	`tests/pagerank/pagerank_10000.txt`	10,000
5	`tests/pagerank/pagerank_50000.txt`	50,000

For example:

```
1
3
```

means:

```
Vertex Coloring
Test case 3
tests/color/color_10000.txt
```

and:

```
2
4
```

means:

```
PageRank
Test case 4
tests/pagerank/pagerank_10000.txt
```

---

4. Graph Input Format

The graph files are stored as adjacency lists.

The first line is:

```
V E
```

where:

- `V` = number of vertices
- `E` = number of edges

Each vertex then has a row:

```
u degree neighbor1 neighbor2 ...
```

Vertices use the convention:

```
0, 1, 2, ..., V-1
```

---

5. CSR Representation

CSR means **Compressed Sparse Row**.

The shared graph layer stores the CSR graph using:

```
struct CSRGraph {
    int V;
    vector<int> offset;
```

```
    vector<int> to;
};
```

For vertex `u`, its neighbours are located between:

```
offset[u]
```

and:

```
offset[u + 1]
```

in the `to` array.

The conversion is performed by:

```
convertToCSR(...)
```

in:

```
src/graph_io.cpp
```

The driver performs this conversion before calling either algorithm.

Timing rule

CSR conversion is preprocessing and is **not included** in algorithm execution time.

The order is:

```
Read input
     ↓
Validate input
     ↓
Convert adjacency list → CSR
     ↓
START TIMER
     ↓
Run algorithm
     ↓
STOP TIMER
```

```
      ↓
Print result
```

---

6. Vertex Coloring

6.1 Objective

Vertex Coloring assigns a color to every vertex so that no two adjacent vertices have the same color.

Finding the true minimum chromatic number is NP-hard, so this implementation uses the required greedy heuristic.

6.2 Welsh–Powell Ordering

The implementation first computes:

```
degree(u) = offset[u + 1] - offset[u]
```

for every vertex.

Vertices are sorted by **non-increasing degree**.

For equal degrees, the vertex number is used as a deterministic tie-breaker.

6.3 Greedy Assignment

For every vertex in Welsh–Powell order:

1. Inspect its already-colored neighbours.
2. Record their colors.
3. Start from color `0`.
4. Select the smallest color not used by those neighbours.
5. Assign that color.

Example:

```
Used neighbour colors = {0, 1, 3}
```

The smallest available color is:

```
2
```

so the vertex receives color `2` .

6.4 Validity

After coloring, every edge is checked.

For an edge:

```
u -- v
```

the condition must be:

```
color[u] != color[v]
```

If every edge satisfies this:

```
Valid: true
```

6.5 Output

The driver prints:

```
Algorithm: Greedy Vertex Coloring
Test case: ...
Input: ...
Vertices: ...
Edges: ...
Vertex colors:
...
Colors used: ...
Valid: true
Execution time: ... ms
```

The exact color numbers are not expected to match a single fixed answer because several valid colorings can exist.

---

7. PageRank

7.1 Objective

PageRank estimates the importance of vertices in a directed graph.

The update formula is:

```
PR(v) = (1-d)/N
        + d * Σ(PR(u) / outdegree(u))
```

where:

- `N` = number of vertices
- `d` = damping factor
- `u -> v` is an incoming edge

The recommended damping factor is:

```
0.85
```

7.2 Initialization

All vertices start with equal rank:

```
PR(v) = 1/N
```

Therefore the initial rank sum is:

```
1.0
```

7.3 Simultaneous Updates

Each iteration calculates the complete new rank vector from the previous rank vector.

Two vectors are used:

```
rank
next
```

This prevents newly calculated values from affecting other vertices in the same iteration.

7.4 Dangling Vertices

A dangling vertex has:

```
outdegree = 0
```

The implementation does not divide by zero.

Instead, the rank belonging to dangling vertices is accumulated and its damped contribution is distributed uniformly across all vertices.

7.5 Convergence

After each iteration:

```
change = Σ |newRank[i] - oldRank[i]|
```

The algorithm stops when:

```
change <= tolerance
```

or when:

```
MAX_ITERATIONS
```

is reached.

The output reports:

```
Iterations: ...
Converged: true/false
```

7.6 Rank Sum

The driver calculates:

```
Sum of ranks
```

The result should remain approximately:

```
1.0
```

---

8. PageRank Input Format

A PageRank test file has:

```
V E
u outdegree neighbor1 neighbor2 ...
...
DAMPING d
TOLERANCE epsilon
MAX_ITERATIONS n
```

Example:

```
4 4
0 1 1
1 1 2
2 1 0
3 1 2
DAMPING 0.85
TOLERANCE 0.0001
MAX_ITERATIONS 100
```

PageRank graphs are directed, so only outgoing edges are listed.

9. Vertex Coloring Input Rules

For Vertex Coloring:

- The graph is undirected.
- The graph is unweighted.
- Every edge appears in both endpoint adjacency lists.
- Self-loops are rejected.
- Duplicate/parallel edges are rejected.
- Vertex IDs must be within `0 ... V-1`.
- The number of listed adjacency entries must equal `2E`.
- Every listed edge must have its reverse endpoint.

An isolated vertex is allowed.

---

10. PageRank Input Rules

For PageRank:

- The graph is directed.
- Only outgoing edges are listed.
- Vertex IDs must be within `0 ... V-1`.
- The number of directed edges must equal `E`.
- Damping must satisfy:

```
0 < damping < 1
```

- Tolerance must be positive.
- Maximum iterations must be positive.

Dangling vertices are handled by the PageRank implementation.

---

11. Required Input Sizes

11.1 Vertex Coloring

Required:

```
10
100
10,000
50,000
100,000
```

Large graphs are kept sparse.

11.2 PageRank

Required:

```
10
100
1,000
10,000
50,000
```

The 100,000-vertex PageRank case is optional according to the supplied specification and is not part of the five required PageRank test cases here.

---

12. Timing and Measurement

Timing is performed immediately around the algorithm call.

Conceptually:

```
auto start = std::chrono::high_resolution_clock::now();

result = algorithm(...);

auto stop = std::chrono::high_resolution_clock::now();
```

The reported time does **not** include:

- file reading
- input parsing
- validation
- graph allocation/setup
- adjacency-list construction
- adjacency-list → CSR conversion
- result printing
- output-file writing

For Vertex Coloring, the timed section contains the coloring algorithm.

For PageRank, all rank-update iterations are inside the timed section.

Times are reported in milliseconds.

Because execution time depends on the computer and system load, the report should use the actual measured value from the machine on which the final tests are run.

---

13. Building

The project uses C++17.

Build:

```
make
```

The Makefile compiles:

```
driver/main.cpp
src/graph_io.cpp
src/color.cpp
src/pagerank.cpp
```

and produces:

```
a4_driver
```

Clean:

```
make clean
```

Build and run:

```
make run
```

---

14. Running

Run:

```
./a4_driver
```

Vertex Coloring

Select:

```
1
```

Then select a test:

```
1
2
3
4
5
```

No filename is entered.

Example:

```
Enter choice: 1

Vertex Coloring test cases:
1. Test case 1
2. Test case 2
3. Test case 3
4. Test case 4
```

```
5. Test case 5
Enter test case (1-5): 3
```

This automatically runs:

```
tests/color/color_10000.txt
```

PageRank

Select:

```
2
```

Then choose:

```
1-5
```

Example:

```
Enter choice: 2

PageRank test cases:
1. Test case 1
2. Test case 2
3. Test case 3
4. Test case 4
5. Test case 5
Enter test case (1-5): 4
```

This automatically runs:

```
tests/pagerank/pagerank_10000.txt
```

---

15. Required Report File With Result Tables

The following tables are the required report format.

Important: `E` , colors, PageRank values, iterations, and execution times should be filled from the actual test run on the submission machine. Execution time is machine-dependent and should not be invented or treated as a fixed value.

15.1 Vertex Coloring Results Table

File	V	E	Colors Used	Valid?	Time	Status
`color_10.txt`	10	20	4	Yes	Record actual ms	Pass
`color_100.txt`	100	Record	Record	Yes/No	Record actual ms	Pass/Fail
`color_10000.txt`	10,000	Record	Record	Yes/No	Record actual ms	Pass/Fail
`color_50000.txt`	50,000	Record	Record	Yes/No	Record actual ms	Pass/Fail
`color_100000.txt`	100,000	Record	Record	Yes/No	Record actual ms	Pass/Fail

Status

A Vertex Coloring test is `Pass` when:

```
Valid = Yes
```

and the program completes normally.

---

16. PageRank Results Table

File	V	E	Damping	Top Vertex	Sum of Ranks	Iter. / Time	Status
`pagerank_10.txt`	10	20	0.85	Record	~1.000	Record / actual ms	Pass/Fail
`pagerank_100.txt`	100	Record	0.85	Record	~1.000	Record / actual ms	Pass/Fail
`pagerank_1000.txt`	1,000	Record	0.85	Record	~1.000	Record / actual ms	Pass/Fail
`pagerank_10000.txt`	10,000	Record	0.85	Record	~1.000	Record / actual ms	Pass/Fail

File	V	E	Damping	Top Vertex	Sum of Ranks	Iter. / Time	Status
`pagerank_50000.txt`	50,000	Record	0.85	Record	~1.000	Record / actual ms	Pass/Fail

Status

A PageRank test should be marked `Pass` when:

- the input is valid,
- the algorithm completes normally,
- the rank sum is approximately `1.0`,
- the final iteration count is reported,
- and the final convergence status is reported.

If the algorithm reaches `MAX_ITERATIONS` before the tolerance is satisfied, it should report:

```
Converged: false
```

while still reporting the final rank vector and iteration count.

---

17. Complexity

Vertex Coloring

Degree calculation from CSR:

```
O(V)
```

Welsh–Powell ordering:

```
O(V log V)
```

Greedy neighbour processing depends on the graph's adjacency storage and the number of colors used.

For sparse graphs, CSR requires approximately:

```
O(V + E)
```

graph storage.

PageRank

One PageRank iteration processes the CSR graph in approximately:

```
O(V + E)
```

If convergence takes `I` iterations:

```
O(I(V + E))
```

The rank vectors require:

```
O(V)
```

additional storage.

---

18. Real-Life Applications

Vertex Coloring

Vertex Coloring has applications such as:

- compiler register allocation
- exam and course timetabling
- wireless frequency/channel assignment
- map coloring
- constraint problems
- graph-based image segmentation
- graph labeling and feature construction

PageRank

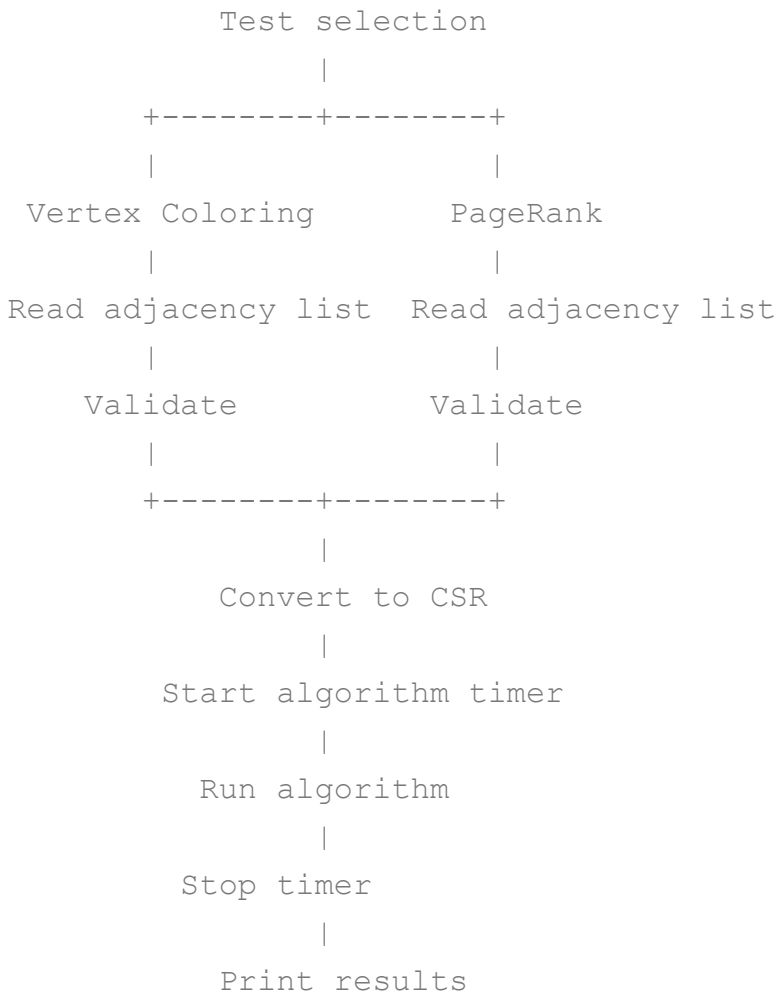
PageRank can be used for:

- search-engine ranking
- social-network influence analysis
- academic citation ranking
- recommendation systems
- extractive text summarization
- graph-based ranking and feature extraction

---

19. Design Summary

The project follows this flow:



The graph representation and CSR conversion are shared by both algorithms.

The algorithm implementation itself is separated into:

```
src/color.cpp
src/pagerank.cpp
```

while the runtime/testing interface is contained in:

```
driver/main.cpp
```
